

Учебный проект на Go

Вторая итерация: каталог, корзина и заказы

Цель итерации

Закончить основную in-memory версию маркетплейса. Добавить товары, корзину, оформление и отмену заказов, а также историю операций.

Продолжаем проект из первой итерации. Пользователи и балансы уже должны работать. Теперь добавляем остальные основные возможности, но пока остаемся только на мапах, слайсах, строках и функциях.

1. Как выполнять итерацию

Не нужно пытаться написать все сразу. Двигайтесь последовательно и после каждого блока проверяйте работу программы.

- 1 Добавьте каталог товаров.
- 2 Добавьте корзину пользователя.
- 3 Реализуйте оформление заказа.
- 4 Добавьте отмену заказа и историю операций.

На этом этапе используем:

- мапы и вложенные мапы;
- слайсы;
- строки;
- функции, циклы и условия;
- стандартные пакеты Go.

Пока не используем:

- структуры и методы;
- интерфейсы и указатели;
- HTTP и базы данных;
- горутины и каналы.

Все данные пока хранятся только в памяти и пропадают после завершения программы. Денежные значения, как и в первой итерации, храним целыми числами в копейках.

2. Новые данные

К существующим мапам `users` и `balances` добавьте:

```
productNames := map[int]string{}
productPrices := map[int]int{}
productStocks := map[int]int{}

carts := map[int]map[int]int{}

orderOwners := map[int]int{}
orderItems := map[int]map[int]int{}
orderItemPrices := map[int]map[int]int{}
orderTotals := map[int]int{}
orderStatuses := map[int]string{}

operationHistory := map[int][]string{}
```

- `productNames` хранит названия товаров;
- `productPrices` хранит цены товаров;
- `productStocks` хранит количество товаров на складе;
- `carts` хранит товары и количество для каждого пользователя;
- `orderItems` хранит состав каждого заказа;
- `orderItemPrices` хранит цены товаров на момент оформления;
- `orderStatuses` хранит статус заказа;
- `operationHistory` хранит историю операций пользователя.

3. Каталог товаров

Реализуйте функции:

```
func AddProduct(
    names map[int]string, prices map[int]int, stocks map[int]int,
    id int, name string, price int, stock int,
) bool

func GetProduct(
    names map[int]string, prices map[int]int, stocks map[int]int,
    productID int,
) (string, int, int, bool)

func UpdateProductStock(
    names map[int]string, stocks map[int]int,
    productID int, stock int,
) bool

func SearchProducts(names map[int]string, query string) []int
```

3. Каталог товаров, продолжение

Что должно работать:

- Идентификатор товара должен быть больше нуля и не должен повторяться.
- Название товара после удаления пробелов по краям не должно быть пустым.
- Цена должна быть больше нуля.
- Количество на складе не может быть отрицательным.
- Получение товара должно возвращать название, цену, остаток и признак существования.
- Поиск выполняется по части названия и не учитывает регистр.

4. Корзина

В мапе carts ключ первого уровня является идентификатором пользователя. Во вложенной мапе ключом является идентификатор товара, а значением - его количество.

```
carts[userID][productID] = quantity
```

Реализуйте функции:

```
func AddToCart(
    users map[int]string, products map[int]string,
    carts map[int]map[int]int,
    userID int, productID int, quantity int,
) bool

func SetCartQuantity(
    carts map[int]map[int]int,
    userID int, productID int, quantity int,
) bool

func RemoveFromCart(
    carts map[int]map[int]int, userID int, productID int,
) bool

func ClearCart(carts map[int]map[int]int, userID int) bool

func CalculateCartTotal(
    carts map[int]map[int]int, prices map[int]int, userID int,
) (int, bool)
```

4. Корзина, продолжение

Что нужно учесть:

- Добавить товар можно только существующему пользователю.
- Добавляемый товар должен существовать.
- Количество должно быть больше нуля.
- Если товар уже есть в корзине, его количество увеличивается.
- Если в SetCartQuantity передано количество 0, товар удаляется.
- При добавлении товара в корзину остаток на складе не изменяется.
- Проверка остатка выполняется при оформлении заказа.

5. Оформление заказа

Реализуйте функцию:

```
func Checkout(  
    users map[int]string,  
    balances map[int]int,  
    productPrices map[int]int,  
    productStocks map[int]int,  
    carts map[int]map[int]int,  
    orderOwners map[int]int,  
    orderItems map[int]map[int]int,  
    orderItemPrices map[int]map[int]int,  
    orderTotals map[int]int,  
    orderStatuses map[int]string,  
    operationHistory map[int][]string,  
    userID int,  
) (int, bool)
```

**Большая
сигнатура**

На этом этапе это нормально. Позже связанные данные будут собраны в структуры, и код станет значительно удобнее.

5. Оформление заказа, продолжение

Функция должна последовательно:

- 1 Проверить существование пользователя.
- 2 Проверить, что корзина существует и не пустая.
- 3 Проверить существование всех товаров из корзины.
- 4 Проверить количество каждого товара на складе.
- 5 Рассчитать итоговую стоимость.
- 6 Проверить баланс пользователя.
- 7 Создать новый идентификатор заказа.
- 8 Скопировать товары и их количество из корзины в заказ.
- 9 Сохранить цены товаров на момент оформления.
- 10 Списать деньги с баланса.
- 11 Уменьшить остатки товаров.
- 12 Сохранить владельца, сумму и статус paid.
- 13 Очистить корзину.
- 14 Добавить запись в историю операций.

Главное условие

Сначала выполните все проверки и только потом изменяйте данные. Если оформление невозможно, баланс, остатки, корзина и заказы не должны измениться.

Копирование корзины

Заказ должен хранить отдельную копию корзины. После очистки или изменения корзины состав уже созданного заказа не должен меняться.

6. Отмена заказа

Реализуйте функцию:

```
func CancelOrder(
    balances map[int]int, productStocks map[int]int,
    orderOwners map[int]int, orderItems map[int]map[int]int,
    orderTotals map[int]int, orderStatuses map[int]string,
    operationHistory map[int][]string,
    userID int, orderID int,
) bool
```

6. Отмена заказа, продолжение

- 1 Проверить существование заказа.
 - 2 Проверить, что заказ принадлежит пользователю.
 - 3 Проверить, что заказ имеет статус paid.
 - 4 Вернуть товары на склад.
 - 5 Вернуть пользователю полную стоимость заказа.
 - 6 Изменить статус заказа на cancelled.
 - 7 Добавить запись в историю операций.
- Чужой заказ отменить нельзя.
 - Уже отмененный заказ нельзя отменить повторно.
 - Деньги и товары нельзя вернуть два раза.

7. Получение информации

Реализуйте функции:

```
func GetUserOrders(
    orderOwners map[int]int, userID int,
) []int

func GetOrderItems(
    orderItems map[int]map[int]int, orderID int,
) (map[int]int, bool)

func GetUserHistory(
    history map[int][]string, userID int,
) []string
```

Возвращайте копии внутренних мап и слайсов. Изменение полученного результата не должно менять данные внутри проекта.

8. Проверка работы

- 1 Добавьте несколько пользователей и товаров.
- 2 Выполните поиск товаров по части названия.
- 3 Добавьте один товар в корзину несколько раз.
- 4 Измените количество товара и посчитайте стоимость корзины.
- 5 Пополните баланс и успешно оформите заказ.
- 6 Проверьте новый баланс, остатки, заказ и пустую корзину.
- 7 Попробуйте оформить заказ при недостаточном балансе.
- 8 Попробуйте купить больше товара, чем есть на складе.
- 9 Отмените оплаченный заказ и проверьте возврат.
- 10 Попробуйте отменить тот же заказ повторно.

9. Коммиты

Желательно разделять функциональность на отдельные коммиты, чтобы по истории было видно, как постепенно развивался проект.

Например:

```
add product catalog
add cart operations
add order checkout
add order cancellation
add operation history
```

Названия могут быть другими. Главное, чтобы один коммит содержал одно понятное изменение.

10. Дополнительные задания

- 1 Найти три самых продаваемых товара.
- 2 Посчитать общую сумму покупок пользователя.
- 3 Вывести товары, которые скоро закончатся.
- 4 Добавить простой промокод на скидку.
- 5 Запретить одному пользователю применять промокод повторно.

Результат

После второй итерации должна получиться рабочая консольная версия маркетплейса: пользователи, балансы, товары, корзины, заказы, отмена и история операций.